CS373 IDB1

House Downing

Charlina Hung, Danish Jaffer, Mallory Koai, Oliver Lee, Ariel Sibille, Troy Varney

1. Introduction

Problem & Use Cases

The purpose of this website is to create and display a structured representation of information about the complex Game of Thrones universe to users. This television series is known for its rich and myriad lore. There are many characters, a wide range of locations, a diverse array of influential organizations, and a number of important events that occur throughout the show. It is no surprise that watchers often have difficulty keeping track of everything. This website is meant to aid users in keeping track of this information by allowing them to search for pertinent data on characters, organizations, events, and locations.

2. Models

UML Class Diagram

There are four primary models: character, organization, location, and event. There are one-to-one, one-to-many, and many-to-many relationships between these models. Each row in each model can be uniquely referenced by an integer ‘id’ primary key.

Character Model

The ‘Character’ model has a many-to-many relationship with every other primary model. Every character can visit many locations, affiliate with many organizations, and participate in many events. These many-to-many relationships are represented by the junction

tables ‘Visitor’, ‘Membership’, and ‘Participant’ respectively. The ‘Character’ model further defines two one-to-one relationships with ‘Location’: location of birth and location of death. Attributes of a character include the id, first and last name, gender, religion, status, and description of the character.

Organization Model

The ‘Organization’ model shares a many-to-many relationship with the ‘Event’ and ‘Character’ models. Any event can involve many organizations, and an organization can partake in many events. Furthermore, an organization can have many members, and a character can be affiliated with many organizations. The junction tables ‘Membership’ and ‘Party’ represent these many-to-many relationships. The ‘Organization’ model also has two foreign keys: one pointing to a location, indicating the organization’s base location, and one pointing to a character, indicating the organization’s leader. Attributes of an id, organization include the name, type, and description of the organization.

Location Model

The ‘Location’ model has a many-to-many relationship to the ‘Character’ model. It also has a one-to-one relationship to itself, where any location has a superior location. Attributes of a location include the id, name, location type, and description of the location.

Event Model

The ‘Event’ model has many-to-many relationships with the ‘Character’ and ‘Organization’ models as described above. It also has a foreign key ‘loc_id’, indicating where

the event took place. Attributes of an event include the id, name, type, and description of the event.

3. RESTful-API

Currently, there are 4 API endpoints:

GET

/api/characters

ex.

[

{ "character_id" : 1,

"character_firstname" : "Robert",

"character_lastname" : "Baratheon",

"character_gender" : "Male",

"character_religion": "Faith of the Seven",

"character_status": "Deceased",

"character_description": "..." }

,

{ "character_id" : 2,

"character_firstname" : "Cersei",

"character_lastname" : "Lannister",

"character_gender" : "Female",

"character_religion": null,

"character_status": "Alive",

"character_description": "..."},

….

]

GET

/api/organizations

ex.

[

{"org_id" : 1,

"org_name" : "House Arryn",

"leader_id" : null,

"org_type" : "House",

"loc_id" : 5,

"org_description" : "...."},

…

]

GET

/api/locations

ex.

[

{"location_id" : 1,

"location_name" : "Braavos",

"location_superior" : "Free Cities",

"location_type" : "City",

"organization_id" : null,

“location_description" : "....."},

{“location_id" : 2,

“location_name" : "Casterly Rock",

“location_superior" : "Westerlands",

“location_type" : "House Seat",

“organization_id" : 6,

“location_description" : "...."},

….

]

GET

/api/events

ex.

[

{"event_id" : 1,

“event_name" : "The Battle of Blackwater Bay",

“event_type" : "Battle",

"location_id" : 7,

"event_description" : "....."}

,

{"event_id" : 2,

"event_name" : "The Battle of Castle Black",

"event_type" : "Battle",

"location_id" : 3,

"event_description" : ""},

…

]

Each endpoint returns a JSON list of JSON objects. The JSON objects in the list each represent one character, organization, location, or event respectively, and each object will have a number of fields.

Eventually, we will allow the API user to get each character, organization, location, or event individually, parameterized by the object's ID. For example, a GET on /api/events/1 will return only the following JSON object:

{"event_id" : 1,

"event_name" : "The Battle of Blackwater Bay",

"event_type" : "Battle",

"location_id" : 7,

"event_description" : "....."}

This feature however is not yet implemented.

Additionally, our API currently runs on Apiary, not off of our own server. The domain name to use is: http://private-e1914-valarmorghulis.apiary-mock.com. Eventually when we write up the API on our own server, this will be our own website that runs on Rackspace.

4. Front-End Framework

Application Template

The website template was created using HTML, CSS, and the Bootstrap framework. On each page, there is a persistent navigation bar on the top of the page, along with large banners for the splash page and each of the pillars. Bootstrap's convenient gridding system was used to aesthetically display the pillars and their items in rows of three circles. There is breadcrumb on each page to help indicate which page the user is on. The templates for each pillar item are dynamically populated with Javascript and displays an image, its attributes, description, and a link to the source of information. The About page has an title panel that has a sub-panel, introducing the tools that the team used. Under the intro panel, two rows of three members each displayed, separated into front-end and back-end teams. The circles of the team members can be clicked into to view more details. Details include: Bio, Responsibilities, and Stats(Number of commits and issues).

Setup

Currently imported css and js files via CDN:

shift.css (codecademy font)

boostrap.min.css

boostrap-theme.min.css

jquery.min.js

bootstrap.min.js

Local css and js files:

housedowning.css

business-frontpage.css

The JavaScript files are imported at the bottom of layout.html so the page can load faster and to ensure that the DOM loads before any JavaScript action is executed.

Template Inheritance

Under the ‘templates’ directory, ‘layout.html’ defines a persistent navbar and links to the necessary .css and .js resources. Every other template extends ‘layout.html’ and is characterized by a business-header (akin to Bootstrap’s Jumbotron) with a tagline and background corresponding to the page’s content.

Routing Functions with Flask

The functions in ‘FlaskApp.py’ are responsible for generating and then serving the web application’s pages. This is accomplished through Flask’s render_template() method which accepts a template and any number of keyword arguments to be passed to the template. Each function is further bound to a URL by a route() decorator. The URL can also be made dynamic by allowing variable parts delimited by angle brackets. The current application utilizes this by accepting an integer id in the URL that is then passed by name to the resulting page. For example:

@app.route('/characters/<char_id>')

The char_id is an integer that corresponds to a primary key in the active model and is used to correctly index into a JSON object obtained from an API GET request. Note that the JSON

response begins indexing at 0 while char_id begins at 1. To adjust for this discrepancy, the char_id is decremented by one prior to any interaction with the JSON object.

Each model or pillar has its own template (e.g., ‘characters.html’) where entities are displayed in a grid layout. For the time being, outgoing links to individual character pages are hardcoded with the variable ‘id’. The resulting URL is dynamic (e.g., ‘characters/2’) and the elements on the subsequent page are populated based off of the integer argument ‘id’. The elements being populated are referenced via the DOM with the getElementById() method.

5. Tests

The unit tests are designed to test the SQLAlchemy models. They create an instance of the model and return it and check for accuracy. The unit tests check that user webapp can add and retrieve instances of the model using various filters, and ensure that a retrieval of a non-existent value correctly returns the value “None.”

Environment Setup

The server runs on a machine provided for by Rackspace. The IP address is http://104.130.247.236/. The Rackspace machine runs Ubuntu 14.04 LTS (Trusty Tahr) (PVHVM), has 2 vCPUs, 3.75 GB of RAM, a system disk of 50 GB, a network speed of 312.5 Mb/s.

Upon creating an empty machine, it was first necessary to run: apt-get update && apt-get -y upgrade && reboot. Then I installed git with apt-get install git and pulled the server’s code repository into /home. The folder containing all of the code is called cs-373-idb.

It is required to install Flask and SQLAlchemy with pip3. After this, running the server is accomplished by running python3 app.py (sudo is required if not root).

6. Tools

Project Design and Planning

Trello | https://trello.com/b/GUfjBcIM/got-project

Trello is a flexible and visual way to manage and organize projects. A team was created with a Project board. Within the board, there are lists of roles with their corresponding tasks. The lists are: API, Pillars, Back-end, Front-end, UML Diagram, etc. Trello boards allowed the team to make group decisions with its voting feature. Each card may also be assigned to one or more organization member to alert them of their roles. Some offline meetings were decided on the Trello boards.

Communication

Slack | https://house-downing.slack.com/

Slack is a platform for slick team communication and allows for easy sharing of resources within teams. There are team channels for: #api, #back-end, #front-end, #general, #report, etc. Discussion is modularized, and members can easily reference channels relevant to them. Slack data may also be exported as resource review and access. Searching for resources and discussion is also fast and simple. Slack allows for integration of third-party tools, such as GitHub, into the team channels, letting the channel automatically pull activity and information from the tool.

Version Control & Issue Tracking

GitHub | https://github.com/QuiteStochastic/cs373-idb

GitHub is a web-based Git repository service. It is primarily being used as version control for the source code and issue tracking for bugs and implementation. Github allows all members to access and modify the source code from any location. The graphical user interface of the website allows other developers to easily view stats and information about the repository, such as programming language written in, commits, changes, issues, and contributors. The API created on Apiary is connected to the GoT Project repository. The project currently only works on the master branch. There is a .gitignore file in the root folder that hides the excess files were produced from compiling the code.

Nitrous Environment Setup

- **What is NITROUS.IO?**

  Nitrous could be classified Platform as a Service (PaaS). It is used for web application development. It helps software developers by providing them a development environment. The environment can be accessed via web or desktop application.

- **Why use NITROUS.IO?**

  Nitrous provides rich features for the development environment. This takes overhead redundant work out the developers responsibility.

- No matter what you are using Windows machine, a Mac, or an iPad, have Linux Environment always at fingertip. Windows machine, a Mac, or an iPad,
- Create a box in Python/Django, Node.js, or GO language/frameworks in 60 seconds.
- Built-in editor makes easier to create or add files. No hassle of pulling and pushing through FTP or git.
- Provides public URL to easily share your web app on a public URL while it is running.
- Allows collaborators which makes centralized box for the development.
- Provides collaboration mode which allow other users to see what changes are being made. Best option for distance pair programming.
- Autoparts to install databases and tools in seconds, like PostgreSQL, Redis, Node.JS. Meteor, PHP, Apache, and many more.
- TMUX pre-installed to allow the web app run continuously after exiting session.

Suggested Resources

Collaboration Guide: http://help.nitrous.io/collab/

Autoparts Guide: http://help.nitrous.io/autoparts/

Video Introduction: https://learnable.com/hub/play/47

Getting Started: http://help.nitrous.io/categories/getting-started/

Best Practices: http://help.nitrous.io/best-practices/

Front End Architecture Testing Structure



